

Hub de Estudos

Apostilas, exercícios e simulados

</> HTML

{ ; } CSS

JS JavaScript

Git

Desenvolvimento Web Front-End

HTML, CSS, JavaScript e TypeScript



Desenvolvimento Web Front-End

HTML, CSS, JavaScript e TypeScript

Hub de Estudos

Apostilas, exercícios e simulados

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia.

Desenvolvido por TecPro Tecnologias.

Sumário

Capítulo 1 – Como a Web funciona **4**
1.1 Internet e Web 4
1.2 Cliente e servidor 4
1.3 O endereço: a URL 5
1.4 Do nome ao número: o DNS 6
1.5 O HTTP: pedido e resposta 6
1.6 Front-end e back-end 7
1.7 Como o navegador monta a página 8
1.8 As ferramentas de trabalho 8
1.9 A primeira página 9
Erros Comuns 11

Capítulo 2 – HTML: estrutura, semântica e acessibilidade **12**
2.1 Elementos, tags e atributos 12
2.2 O cabeçalho do documento 12
2.3 Títulos e parágrafos 13
2.4 Texto com significado 14
2.5 Links e caminhos 14
2.6 Imagens 16
2.7 Áudio e vídeo 16
2.8 Listas 17
2.9 Tabelas 18
2.10 HTML semântico 19
2.11 Acessibilidade 21
2.12 Caracteres especiais 22
2.13 A página completa 23
Erros Comuns 24

Capítulo 1 – Como a Web funciona

Antes de escrever a primeira linha de HTML, vale entender o caminho que uma página percorre até aparecer na tela: quem a guarda, como ela é pedida, como chega e o que o navegador faz com ela. Neste capítulo você vai conhecer a diferença entre a Internet e a Web, o modelo cliente-servidor, as partes de um endereço, o DNS, o protocolo HTTP e as etapas que o navegador percorre para desenhar uma página. No final, você vai preparar as ferramentas de trabalho e criar a primeira página, com um arquivo de HTML, um de CSS e um de JavaScript.

1.1 Internet e Web

As duas palavras costumam ser usadas como sinônimos, mas não são a mesma coisa. A **Internet** é a rede: milhões de computadores do mundo inteiro ligados por cabos, fibras e antenas, que trocam dados seguindo um conjunto de regras comum, os protocolos TCP/IP. Ela existe desde os anos 1970 e carrega muitos serviços diferentes: e-mail, mensagens, chamadas de vídeo, jogos.

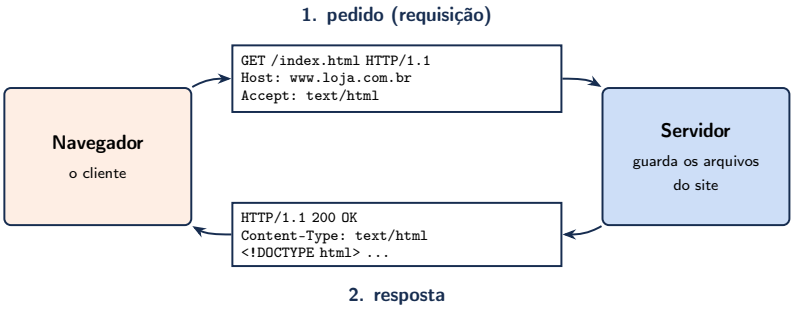
A **Web** (*World Wide Web*) é um desses serviços: um conjunto de documentos, as **páginas**, que ficam guardados em computadores espalhados pela Internet e são ligados uns aos outros por **links**. Ela foi proposta em 1989 por Tim Berners-Lee, um pesquisador do CERN, o laboratório europeu de física de partículas, para que os cientistas compartilhassem documentos. Em 1991, ele publicou o primeiro site, junto com as três peças que sustentam a Web até hoje:

- o **HTML**, a linguagem em que as páginas são escritas;
- a **URL**, o endereço único de cada página;
- o **HTTP**, o protocolo que o navegador usa para pedir uma página e o servidor usa para entregá-la.

As regras dessas tecnologias são definidas por organizações abertas. O **WHATWG**, formado pelas empresas que fazem os navegadores, mantém o HTML como um **padrão vivo** (*living standard*): não existe mais uma versão "final", e o padrão é atualizado continuamente. O nome HTML5, ainda muito usado, se refere à grande atualização publicada em 2014 e virou sinônimo de "HTML moderno". O **W3C** mantém o CSS e outros padrões. Para consultar como cada recurso funciona, a referência mais usada é o **MDN Web Docs** (`developer.mozilla.org`), mantido pela Mozilla, com tradução para o português em boa parte das páginas.

1.2 Cliente e servidor

Toda visita a um site envolve dois papéis. O **cliente** é quem pede: no caso da Web, o **navegador**, como o Chrome, o Firefox, o Edge ou o Safari. O **servidor** é um computador, ligado o tempo todo à Internet, que guarda os arquivos do site e os entrega a quem pedir. A conversa entre os dois é sempre a mesma: o cliente faz um **pedido** (a **requisição**) e o servidor devolve uma **resposta**.



depois, um novo pedido para cada arquivo citado no HTML: CSS, JavaScript, imagens

Uma página quase nunca é um arquivo só. O HTML cita outros arquivos, como folhas de estilo, *scripts* e imagens, e o navegador faz um novo pedido para cada um deles, um atrás do outro ou vários ao mesmo tempo. Uma página de um grande portal chega a fazer centenas de pedidos até ficar pronta.

O servidor pode entregar dois tipos de conteúdo. Um site **estático** é formado por arquivos prontos, iguais para todas as pessoas: o servidor só os encontra e envia. Um site **dinâmico** tem um programa no servidor que monta a resposta na hora, de acordo com quem pede, consultando um banco de dados: é assim que uma loja mostra o seu carrinho e uma rede social mostra o seu *feed*. Nesta apostila, você vai construir a parte que roda no navegador; a parte que roda no servidor é assunto das disciplinas de Web II e Web III.

1.3 O endereço: a URL

Cada recurso da Web tem um endereço, a **URL** (*Uniform Resource Locator*). Uma URL completa tem várias partes:

```
https://www.loja.com.br:443/produtos?busca=caneta#lista
\___/   \______________/ \_/\_______/\___________/\____/
  |            |         |     |          |         |
protocolo   domínio   porta caminho   consulta  fragmento
```

Parte	O que indica
protocolo	as regras da conversa: `https`, ou `http`, sem criptografia
domínio	o nome do servidor, como `www.loja.com.br`
porta	a "porta de entrada" do servidor; quase sempre omitida, porque o `https` usa a 443 e o `http`, a 80
caminho	qual recurso, dentro do servidor, como `/produtos`
consulta	informações extras, em pares `nome=valor`, depois do `?` e separadas por `&`
fragmento	um ponto dentro da página, depois do `#`; ele nem é enviado ao servidor, e só o navegador o usa, para rolar a página até aquele ponto

O próprio JavaScript sabe separar essas partes. Este trecho foi executado no Node.js, que roda JavaScript fora do navegador, com o resultado em comentário:

```
const url = new URL(
  "https://www.loja.com.br:443/produtos?busca=caneta#lista",
);
url.protocol;  // "https:"
url.hostname;  // "www.loja.com.br"
url.port;      // ""  (443 é a porta padrão do https)
url.pathname;  // "/produtos"
url.search;    // "?busca=caneta"
url.hash;      // "#lista"
url.searchParams.get("busca");  // "caneta"
```

Repare na porta vazia: como 443 é a porta padrão do `https`, ela nem precisa aparecer, e o navegador a remove. Os endereços que você vai usar no seu computador, como `http://localhost:3000`, são diferentes: `localhost` é o nome do próprio computador, e a porta precisa aparecer, porque não é a padrão.

1.4 Do nome ao número: o DNS

Na Internet, os computadores se encontram por números, os **endereços IP**, como `104.20.23.154`. Nomes como `example.com` existem para as pessoas. Quem traduz um no outro é o **DNS** (*Domain Name System*), uma espécie de lista telefônica distribuída por servidores do mundo inteiro. O comando `nslookup`, presente no Windows, no Linux e no macOS, faz essa consulta:

```
$ nslookup example.com
...
Nome:    example.com
Addresses:  2606:4700:10::ac42:93f3
            2606:4700:10::6814:179a
            104.20.23.154
            172.66.147.243
```

Um mesmo nome pode ter vários endereços: grandes sites espalham cópias por vários servidores, e o DNS distribui as pessoas entre eles. Os endereços com dois-pontos estão no formato mais novo, o IPv6; os outros, no formato tradicional, o IPv4. O resultado de cada consulta fica guardado por um tempo no seu computador, para que a próxima visita não precise perguntar de novo.

1.5 O HTTP: pedido e resposta

O **HTTP** (*HyperText Transfer Protocol*) define o formato das mensagens trocadas entre o navegador e o servidor. Elas são texto, e dá para vê-las com o `curl`, um programa de linha de comando que faz pedidos HTTP, presente no Windows, no Linux e no macOS. A opção `-I` pede só o cabeçalho da resposta:

```
$ curl -I https://example.com
HTTP/1.1 200 OK
```

```
Date: Sun, 18 Oct 2026 02:01:44 GMT
Content-Type: text/html; charset=utf-8
Server: cloudflare
last-modified: Tue, 13 Oct 2026 20:08:28 GMT
```

A primeira linha traz o **código de status**, um número de três dígitos que resume o resultado, e as seguintes são os **cabeçalhos**, informações sobre a resposta no formato `Nome: valor`. O `Content-Type` é o mais importante para o navegador: `text/html` diz que o conteúdo é uma página; para uma folha de estilo, seria `text/css`, e para uma imagem, `image/png`, por exemplo. Depois dos cabeçalhos vem o **corpo**, com o conteúdo em si, que o `-I` omitiu.

O pedido diz **o que** se quer fazer com um **método**. Os dois que você vai encontrar nesta apostila são o `GET`, para buscar algo, usado em toda página aberta e em todo link clicado, e o `POST`, para enviar dados, usado nos formulários (Capítulo 3). Os códigos de status são agrupados pelo primeiro dígito:

Faixa	Significado	Exemplos comuns
`2xx`	deu certo	`200 OK`
`3xx`	o recurso está em outro lugar	`301 Moved Permanently`, `304 Not Modified`
`4xx`	erro de quem pediu	`404 Not Found`, `403 Forbidden`
`5xx`	erro do servidor	`500 Internal Server Error`

O **HTTPS** é o HTTP protegido por criptografia. Sem ele, qualquer pessoa no caminho, como alguém na mesma rede Wi-Fi de uma lanchonete, pode ler e até alterar o que trafega, inclusive senhas. Com ele, só o navegador e o servidor entendem as mensagens. Hoje praticamente todos os sites usam HTTPS, e os navegadores marcam como "não seguros" os que não usam. Os detalhes do HTTP, como os outros métodos e o uso dos códigos de status em APIs, são estudados a fundo na disciplina de Web III.

1.6 Front-end e back-end

O trabalho de construir um site se divide em duas frentes. O **back-end** é o que roda no servidor: o programa que monta as páginas dinâmicas, guarda os dados em um banco, confere senhas e aplica as regras do negócio. O **front-end** é o que roda no navegador: tudo o que a pessoa vê e com que interage. Esta apostila é sobre o front-end, que se apoia em três linguagens, cada uma com um papel:

Linguagem	Papel	Em uma casa, seria...
HTML	a estrutura e o significado do conteúdo	as paredes, as portas e os cômodos
CSS	a aparência: cores, fontes, espaços e posições	a pintura, os móveis e a decoração
JavaScript	o comportamento: reagir a cliques, buscar dados, mudar a página	a parte elétrica: o interruptor que acende a luz

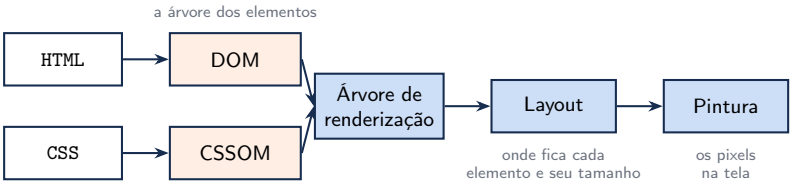
O HTML é uma **linguagem de marcação**: ele descreve o que cada parte do conteúdo é, um título, um parágrafo, uma lista, mas não faz cálculos nem toma decisões. O CSS é uma **linguagem de estilo**. Só o JavaScript é uma **linguagem de programação**,

com variáveis, condições e repetições. Manter cada uma no seu papel, em arquivos separados, deixa o site mais fácil de manter: para mudar a cor de todos os títulos, basta alterar uma linha do CSS, sem mexer no conteúdo.

Mesmo as ferramentas mais modernas, como o React e o Next.js, que você vai estudar na disciplina de Web II, geram HTML, CSS e JavaScript no fim das contas, porque é isso que os navegadores entendem. Quem domina as três linguagens entende o que essas ferramentas produzem e consegue resolver os problemas que elas não resolvem sozinhas.

1.7 Como o navegador monta a página

Entre receber o HTML e mostrar a página, o navegador percorre algumas etapas, chamadas em conjunto de **caminho crítico de renderização**:



1. O navegador lê o HTML e monta o **DOM** (*Document Object Model*), uma árvore em que cada elemento da página é um nó, dentro do seu elemento "pai". É no DOM que o JavaScript mexe para mudar a página (Capítulo 9).
2. Ao encontrar uma folha de estilo, ele a baixa e monta o **CSSOM**, a árvore das regras de estilo.
3. Juntando as duas, ele monta a **árvore de renderização**, só com o que vai aparecer, e já com o estilo de cada elemento.
4. No **layout**, ele calcula a posição e o tamanho de cada elemento, de acordo com a largura da tela.
5. Na **pintura**, ele desenha os pixels: textos, cores, bordas e imagens.

Esse caminho explica algumas práticas que você vai ver ao longo da apostila. Um *script* comum interrompe a leitura do HTML até ser baixado e executado; com o atributo `defer`, ele é baixado em paralelo e só roda quando o HTML terminou de ser lido. E sempre que o JavaScript muda a página, o navegador refaz as etapas de layout e pintura das partes afetadas, o que tem um custo quando é feito muitas vezes seguidas.

1.8 As ferramentas de trabalho

Para desenvolver para a Web, bastam três ferramentas, todas gratuitas:

- um **editor de código**. O mais usado é o **Visual Studio Code** (VS Code), da Microsoft, para Windows, Linux e macOS. Ele colore o código, completa as *tags* enquanto você digita e mostra erros;
- um **navegador moderno**, como o Chrome, o Edge ou o Firefox, com as ferramentas de desenvolvedor;
- um **servidor local**, para abrir as páginas como elas seriam abertas na Web.

As **ferramentas de desenvolvedor** (*DevTools*) abrem com a tecla `F12`, ou com o botão direito do mouse sobre a página, na opção **Inspecionar**. Três abas vão ser usadas o tempo todo:

Aba	Para que serve
Elements	mostra o DOM e o CSS de cada elemento; dá para alterar os dois ali mesmo e ver o efeito na hora, sem mexer nos arquivos
Console	mostra os erros do JavaScript e permite executar comandos
Network	lista cada pedido feito pela página, com o método, o status, o tipo e o tempo

Uma página pode ser aberta com um duplo clique no arquivo, e o endereço passa a começar com `file://`. Para as primeiras páginas, isso funciona, mas alguns recursos, como os pedidos do Capítulo 10, só funcionam quando a página é servida por HTTP. O jeito mais simples de ter um servidor no seu computador é a extensão **Live Server** do VS Code: com ela instalada, o botão **Go Live**, na barra inferior, abre a página no navegador e a recarrega a cada arquivo salvo. Quem já tem o Node.js instalado tem outra opção, sem extensão nenhuma, digitando no terminal, dentro da pasta do site:

```
$ npx serve
 INFO  Accepting connections at http://localhost:3000
```

1.9 A primeira página

Crie uma pasta chamada `primeira-pagina` e, dentro dela, três arquivos. O primeiro é o HTML, com o conteúdo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Minha primeira página</title>
  <link rel="stylesheet" href="estilo.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Olá, Web!</h1>
  <p>Esta página tem três arquivos: HTML, CSS e JavaScript.</p>
  <button type="button" id="botao">Clique aqui</button>
  <p id="mensagem"></p>
</body>
</html>
```

As linhas antes do `<body>` são o "cabeçalho" do documento, com informações para o navegador: o idioma, a codificação dos caracteres, o título que aparece na aba e os arquivos de estilo e de *script* usados. Cada uma delas é explicada no Capítulo

2. O conteúdo visível fica dentro do `<body>`: um título, um parágrafo, um botão e um parágrafo vazio, que vai receber uma mensagem. O segundo arquivo, `estilo.css`, define a aparência:

```
body {
  font-family: system-ui, sans-serif;
  max-width: 40rem;
  margin: 2rem auto;
  padding: 0 1rem;
}

h1 {
  color: #24124f;
}

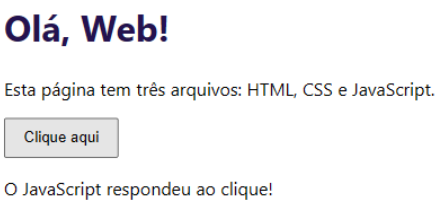
button {
  padding: 0.5rem 1rem;
  cursor: pointer;
}
```

E o terceiro, `script.js`, dá vida ao botão:

```
const botao = document.querySelector("#botao");
const mensagem = document.querySelector("#mensagem");

botao.addEventListener("click", () => {
  mensagem.textContent = "O JavaScript respondeu ao clique!";
});
```

O JavaScript procura o botão e o parágrafo vazio pelos seus `id`, e, a cada clique no botão, escreve uma frase no parágrafo. Abra a página com o Live Server, ou com o `npx serve`, e clique no botão:



Na aba Network das ferramentas de desenvolvedor, recarregue a página. A lista mostra os três pedidos que o navegador fez, um para cada arquivo, cada um com o seu tipo:

```
GET /           200  text/html
GET /estilo.css 200  text/css
GET /script.js  200  text/javascript
```

O primeiro pedido, para `/`, devolveu o `index.html`: quando o caminho é uma pasta, os servidores procuram, por convenção, um arquivo com esse nome dentro dela. Os outros dois foram feitos porque o HTML os citou. Talvez apareça também um quarto pedido, para `/favicon.ico`, com status `404`, e um erro no Console: o navegador procura sozinho o ícone que aparece na aba, e a página ainda não tem um. Esse erro é inofensivo, e o Capítulo 2 mostra como declarar o ícone. Se você errar o nome de um arquivo, como `estilos.css` em vez de `estilo.css`, a página aparece sem estilo, e a aba Network mostra o pedido com o status `404`: é o primeiro lugar a olhar quando algo não carrega.

Por fim, uma página pode estar "funcionando" na tela e, mesmo assim, ter erros que um navegador perdoa e outro não. O **validador do W3C** (`validator.w3.org`) confere o HTML contra o padrão. Esta página foi conferida com ele e com o `html-validate`, outra ferramenta de validação, que acusou um detalhe na primeira versão: o `<button>` estava sem o atributo `type`. Sem ele, um botão dentro de um formulário envia o formulário ao ser clicado, o que nem sempre é o desejado; por isso a página declara `type="button"`, assunto do Capítulo 3.

Erros Comuns

- **Confundir a Internet com a Web.** A Internet é a rede; a Web é um dos serviços que funcionam sobre ela.
- **Achar que o HTML é uma linguagem de programação.** Ele descreve o conteúdo; quem programa o comportamento é o JavaScript.
- **Errar o nome ou a pasta de um arquivo citado no HTML.** O navegador recebe um `404`, e a página aparece sem estilo ou sem *script*. Confira a aba Network.
- **Misturar as três linguagens em um arquivo só.** Estilo e *script* espalhados no HTML tornam qualquer mudança trabalhosa. Separe cada um no seu arquivo.
- **Esquecer o `defer` no *script* do `<head>`.** O *script* roda antes de o HTML existir, e o `querySelector` não encontra o elemento.
- **Testar só abrindo o arquivo com duplo clique.** Alguns recursos só funcionam com a página servida por HTTP. Use um servidor local.
- **Ignorar o Console.** Um erro de JavaScript costuma estar escrito lá, com o arquivo e a linha.

Capítulo 2 – HTML: estrutura, semântica e acessibilidade

O HTML é a base de toda página: é ele que diz o que cada pedaço do conteúdo **é**. Um título, um parágrafo, uma lista de ingredientes, um menu, uma tabela. Neste capítulo você vai conhecer os elementos do HTML, montar o cabeçalho completo de um documento, marcar textos, links, imagens, vídeos, listas e tabelas, e organizar a página com os elementos semânticos. No caminho, vai entender por que um HTML bem escrito é a primeira condição para que pessoas com deficiência consigam usar um site. No final, uma página de receita reúne tudo, conferida por validadores e por um verificador de acessibilidade.

2.1 Elementos, tags e atributos

Um documento HTML é feito de **elementos**. A maioria tem uma **tag** de abertura, um conteúdo e uma **tag** de fechamento, que repete o nome com uma barra. A tag de abertura pode trazer **atributos**, informações extras no formato `nome="valor"`:

```
<a href="bolos.html" title="Ver todas as receitas">Bolos</a>
\_____________________________________________/\___/\__/
                        |                        |    |
        tag de abertura, com dois atributos     texto
                                                   fechamento
```

Alguns elementos não têm conteúdo nem tag de fechamento: são os **elementos vazios**, como a imagem, a quebra de linha (`<br>`) e os do cabeçalho (`<meta>`, `<link>`). E há os **comentários**, que o navegador ignora, úteis para deixar notas no código:

```
<a href="bolos.html" title="Ver todas as receitas">Bolos</a>
<img src="img/bolo.svg" alt="Bolo de cenoura">
<!-- Um comentário: o navegador ignora, e a página não mostra -->
```

Algumas regras valem para todo o HTML:

- **os elementos se aninham**, um dentro do outro, como caixas: o que abre por último fecha primeiro. `<p><strong>Olá</strong></p>` está certo; `<p><strong>Olá</p></strong>` está errado;
- **espaços e quebras de linha seguidos viram um espaço só**. Apertar `Enter` várias vezes no código não cria linhas em branco na página; quem cuida do espaçamento é o CSS;
- os nomes de tags e atributos não diferenciam maiúsculas de minúsculas, mas a convenção é escrever tudo em minúsculas;
- os valores dos atributos ficam entre aspas duplas.

2.2 O cabeçalho do documento

Todo documento HTML tem a mesma estrutura de base: a declaração do tipo, o elemento `<html>`, que envolve tudo, e, dentro dele, o `<head>`, com informações sobre a

página, e o `<body>`, com o conteúdo visível. Este é o cabeçalho da página de receita que fecha este capítulo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Bolo de cenoura | Receitas da Casa</title>
  <meta name="description"
        content="Bolo de cenoura com cobertura de chocolate.">
  <link rel="icon" href="favicon.svg" type="image/svg+xml">
</head>
```

Linha	Para que serve
`<!DOCTYPE html>`	diz que o documento segue o HTML atual. Sem ela, o navegador entra no "modo de compatibilidade" (*quirks mode*) e imita defeitos de navegadores dos anos 1990
`<html lang="pt-BR">`	o idioma da página: o leitor de tela pronuncia o texto em português, e o tradutor do navegador sabe de que língua partir
`<meta charset>`	a codificação dos caracteres; o UTF-8 representa qualquer letra de qualquer idioma, inclusive ç, ã e emojis
`<meta name="viewport">`	em celulares, usa a largura real da tela; sem ela, a página é desenhada com cerca de 980 pixels de largura e encolhida (Capítulo 6)
`<title>`	o texto da aba e do título nos resultados dos buscadores
`<meta name="description">`	um resumo da página, que os buscadores costumam mostrar abaixo do título
`<link rel="icon">`	o ícone da aba, o *favicon*

O *favicon* resolve o erro que apareceu no Console no Capítulo 1: sem um ícone declarado, o navegador procura sozinho o arquivo `/favicon.ico`. Aqui ele é um SVG, uma imagem vetorial, que os navegadores atuais aceitam e que fica nítida em qualquer tamanho. O `<link rel="stylesheet">` e o `<script defer>`, que você viu no Capítulo 1, também ficam no `<head>`; esta página ainda não tem CSS nem JavaScript.

2.3 Títulos e parágrafos

O HTML tem seis níveis de título, de `<h1>` a `<h6>`. Eles formam um **sumário** da página, e é assim que leitores de tela e buscadores a entendem: o `<h1>` é o assunto da página, cada `<h2>` é uma seção dele, cada `<h3>` é uma subdivisão do `<h2>` anterior, e assim por diante. Três regras:

- use um único `<h1>` por página, com o assunto principal. O padrão não proíbe mais de um, mas um só deixa o sumário claro;
- não pule níveis: depois de um `<h2>` vem um `<h3>`, e não um `<h4>`;

- escolha o nível pela **hierarquia**, e nunca pelo tamanho da letra. Se o título ficou grande demais, quem muda isso é o CSS.

O `<p>` marca um parágrafo. O `<br>` força uma quebra de linha dentro dele, útil em endereços e poemas, mas não deve ser usado para criar espaço entre blocos. O `<hr>` marca uma mudança de assunto entre parágrafos e aparece como uma linha horizontal.

2.4 Texto com significado

Dentro de um parágrafo, alguns elementos dizem algo sobre um trecho do texto:

```
<p>O forno deve estar <strong>preaquecido</strong>.</p>
<p>A massa fica <em>muito</em> mais fofa peneirada.</p>
<p>Use <mark>fermento químico</mark>, e não biológico.</p>
<p>O <abbr title="Instituto Nacional de Metrologia">Inmetro</abbr>
  aprova a forma.</p>
<p>Salve com <kbd>Ctrl</kbd> + <kbd>S</kbd>.</p>
<p>A função <code>alert()</code> mostra um aviso.</p>
<p>Água é H<sub>2</sub>O; a área é 10 m<sup>2</sup>.</p>
<p>Pronto às <time datetime="15:30">três e meia</time>.</p>
<blockquote cite="https://pt.wikipedia.org/wiki/Bolo">
  <p>Bolo é uma massa assada, geralmente doce.</p>
</blockquote>
<p>Como diz o ditado, <q>quem espera sempre alcança</q>.</p>
```

O `<strong>` marca algo **importante**, e o `<em>`, uma **ênfase**, que muda o sentido da frase, como a palavra "muito". O `<mark>` destaca um trecho, o `<abbr>` explica uma sigla no atributo `title`, o `<kbd>` representa teclas, o `<code>`, um trecho de código, e o `<sub>` e o `<sup>`, índices e expoentes. O `<time>` traz, no atributo `datetime`, a data ou a hora em um formato que programas entendem, enquanto o texto fica livre para as pessoas. O `<blockquote>` é uma citação longa, em bloco, e o `<q>`, uma citação curta, no meio da frase.

Existem também o `<b>` e o `<i>`, que, por padrão, também deixam o texto em negrito e em itálico. A diferença é o significado:

Elemento	Quando usar
`<strong>`	o trecho é importante, sério ou urgente
`<b>`	destaque só visual, sem importância especial, como uma palavra-chave em um resumo
`<em>`	ênfase, que muda o sentido da frase
`<i>`	uma voz diferente do texto: termos estrangeiros, nomes científicos, títulos de obras

Na dúvida, prefira os que carregam significado, o `<strong>` e o `<em>`. E, para deixar um texto em negrito só porque "fica bonito", não use nenhum dos quatro: use o CSS.

2.5 Links e caminhos

O link é o elemento que fez a Web: o `<a>`, de *anchor* (âncora), com o destino no atributo `href`:

```
<a href="https://developer.mozilla.org">MDN Web Docs</a>
<a href="bolos.html">Bolos</a>
<a href="receitas/salgados.html">Salgados</a>
<a href="../index.html">Início</a>
<a href="#ingredientes">Ir para os ingredientes</a>
<a href="mailto:contato@receitas.com.br">Escreva para nós</a>
<a href="tel:+5521999990000">(21) 99999&#8209;0000</a>
<a href="cardapio.pdf" download>Baixar o cardápio</a>
<a href="https://www.w3.org" target="_blank">W3C (nova aba)</a>
```

O destino pode ser escrito de dois jeitos. Um **endereço absoluto** é completo, com protocolo e domínio, e é usado para outros sites. Um **endereço relativo** é escrito a partir da página atual e é usado dentro do próprio site. Imagine esta estrutura de pastas:

```
site/
  index.html
  receitas/
    bolo.html          <- a página atual
    bolos.html
    salgados.html
  img/
    bolo.svg
```

Em `receitas/bolo.html`	Aponta para
`bolos.html`	um arquivo na mesma pasta
`../index.html`	um arquivo na pasta de cima: `..` sobe um nível
`../img/bolo.svg`	sobe um nível e entra na pasta `img`
`/index.html`	começa pela raiz do site, de qualquer página

Os endereços relativos continuam funcionando quando o site muda de domínio ou é copiado para outra pasta, por isso são a regra dentro do site. O `#ingredientes` leva a um ponto da própria página: o elemento que tem `id="ingredientes"`. O `mailto:` abre o programa de e-mail, e o `tel:` oferece a ligação no celular. O atributo `download` pede que o arquivo seja baixado, em vez de aberto.

O `target="_blank"` abre o link em uma nova aba. Até 2020, era preciso acrescentar `rel="noopener"` por segurança, porque a página aberta conseguia controlar a aba de origem; os navegadores atuais já aplicam essa proteção sozinhos, e o atributo só é necessário para quem precisa atender navegadores muito antigos. Use o `_blank` com moderação: abrir abas sem pedir confunde muita gente, principalmente quem usa leitor de tela.

O texto do link deve dizer para onde ele leva, mesmo lido fora do contexto. Leitores de tela oferecem uma lista com todos os links da página, e uma lista com dez "clique aqui" não ajuda ninguém. Prefira "Baixar o cardápio" a "Para baixar o cardápio, clique aqui".

2.6 Imagens

A imagem é um elemento vazio, `<img>`, com o arquivo no atributo `src`. Na página de receita:

```
<figure>
  <img src="img/bolo.svg" width="320" height="200"
       alt="Bolo retangular com cobertura de chocolate">
  <figcaption>O bolo pronto, ainda na forma.</figcaption>
</figure>
```

O `alt` é o texto alternativo: é o que o leitor de tela lê no lugar da imagem, o que aparece se ela não carregar e o que os buscadores usam para entendê-la. Ele é obrigatório, e o validador do W3C acusa a falta dele. Escreva o que a imagem **mostra** e que importa para o conteúdo: "Bolo retangular com cobertura de chocolate", e não "imagem" ou "bolo.svg". Se a imagem é só decorativa, como um enfeite, use `alt=""`, vazio: assim o leitor de tela sabe que pode pulá-la.

A `width` e a `height` dão o tamanho original da imagem. Com elas, o navegador reserva o espaço antes de baixá-la, e o texto não "pula" quando ela chega. O atributo `loading="lazy"` adia o carregamento das imagens que estão longe da área visível, até a pessoa rolar a página até elas, o que economiza dados no celular. O `<figure>` agrupa a imagem com uma legenda, o `<figcaption>`. Os formatos mais usados:

Formato	Melhor para
JPEG	fotografias
PNG	ilustrações, capturas de tela e imagens com transparência
SVG	ícones, logotipos e desenhos: é vetorial, fica nítido em qualquer tamanho e é escrito em texto
WebP e AVIF	os formatos modernos: a mesma qualidade do JPEG e do PNG, com arquivos bem menores

2.7 Áudio e vídeo

O HTML toca áudio e vídeo sem nenhum programa extra:

```
<video controls width="640" height="360" preload="metadata"
       poster="img/capa-video.jpg">
  <source src="video/preparo.webm" type="video/webm">
  <source src="video/preparo.mp4" type="video/mp4">
  <track kind="captions" src="video/preparo.vtt"
         srclang="pt" label="Português" default>
  <a href="video/preparo.mp4">Baixe o vídeo do preparo</a>.
</video>

<audio controls src="audio/dica.mp3">
  <a href="audio/dica.mp3">Ouça a dica</a>.
</audio>
```

O `controls` mostra os botões de tocar, pausar e volume. Cada `<source>` oferece o mesmo vídeo em um formato, e o navegador usa o primeiro que conseguir tocar. O

`poster` é a imagem exibida antes de o vídeo começar, e o `preload="metadata"` baixa só as informações do vídeo, como a duração, até a pessoa dar o *play*. O `<track>` acrescenta legendas, em um arquivo de texto no formato WebVTT: elas são indispensáveis para pessoas surdas e úteis para quem assiste sem som. O conteúdo que sobra dentro do `<video>`, o link, só aparece em navegadores que não tocam vídeo. Evite o `autoplay`: um som que começa sozinho atrapalha qualquer pessoa, e atrapalha muito quem usa leitor de tela.

2.8 Listas

O HTML tem três tipos de lista. A **não ordenada**, `<ul>`, é para itens em que a ordem não importa, como os ingredientes; a **ordenada**, `<ol>`, para passos em sequência, como o modo de preparo. Cada item é um `<li>`:

```
<section>
  <h2>Ingredientes</h2>
  <ul>
    <li>3 cenouras médias</li>
    <li>3 ovos</li>
    <li>1 xícara de óleo</li>
    <li>2 xícaras de açúcar</li>
    <li>2 xícaras de farinha de trigo</li>
    <li>1 colher (sopa) de fermento em pó</li>
  </ul>
</section>

<section>
  <h2>Modo de preparo</h2>
  <ol>
    <li>Bata no liquidificador as cenouras, os ovos
      e o óleo.</li>
    <li>Misture o açúcar e a farinha, e por último
      o fermento.</li>
    <li>Asse em forno médio, a 180 °C, por cerca
      de 40 minutos.</li>
  </ol>
  <p><em>Não</em> abra o forno nos primeiros 25 minutos.</p>
</section>
```

Uma lista pode ficar dentro de um item de outra, formando sublistas. Os menus de navegação também são listas: uma lista de links, como no cabeçalho da página de receita (Seção 2.10). O leitor de tela anuncia "lista com 6 itens" antes de lê-los, o que ajuda a pessoa a saber o que vem pela frente. O terceiro tipo, a **lista de definições**, `<dl>`, reúne pares de termo (`<dt>`) e descrição (`<dd>`), como em um glossário:

```
<dl>
  <dt>Banho-maria</dt>
  <dd>Cozinhar dentro de um recipiente com água quente.</dd>
  <dt>Untar</dt>
  <dd>Passar gordura na forma para a massa não grudar.</dd>
</dl>
```

2.9 Tabelas

Tabelas são para **dados em linhas e colunas**, em que cada célula se relaciona com o título da sua linha e o da sua coluna: notas, horários, preços, comparações. A tabela de informação nutricional da receita:

```
<section>
  <h2>Informação nutricional</h2>
  <table>
    <caption>Valores aproximados por fatia</caption>
    <thead>
      <tr>
        <th scope="col">Item</th>
        <th scope="col">Quantidade</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">Calorias</th>
        <td>310 kcal</td>
      </tr>
      <tr>
        <th scope="row">Carboidratos</th>
        <td>45 g</td>
      </tr>
    </tbody>
  </table>
</section>
```

O `<caption>` é o título da tabela. O `<thead>` e o `<tbody>` separam o cabeçalho do corpo; existe também o `<tfoot>`, para totais. Cada `<tr>` é uma linha. As células de dados são `<td>`, e as de título, `<th>`, com o atributo `scope`, que diz se o título é da coluna (`col`) ou da linha (`row`). Com isso, ao chegar à célula "310 kcal", o leitor de tela consegue anunciar "Calorias, Quantidade: 310 kcal". Uma célula pode ocupar mais de uma coluna, com o `colspan`, ou mais de uma linha, com o `rowspan`:

```
<table>
  <caption>Horário das aulas</caption>
  <thead>
    <tr>
      <th scope="col">Horário</th>
      <th scope="col">Segunda</th>
      <th scope="col">Terça</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">19h</th>
      <td rowspan="2">Confeitaria</td>
      <td>Massas</td>
    </tr>
    <tr>
```

```
      <th scope="row">20h</th>
      <td>Molhos</td>
    </tr>
    <tr>
      <th scope="row">21h</th>
      <td colspan="2">Prática livre</td>
    </tr>
  </tbody>
</table>
```

A “Confeitaria” ocupa as linhas das 19h e das 20h na segunda-feira, por isso a linha das 20h tem uma célula a menos; a “Prática livre” ocupa as duas colunas da linha das 21h. Nunca use tabelas para posicionar o conteúdo da página, um costume dos anos 1990: elas se comportam mal em telas pequenas e confundem os leitores de tela, que anunciam linhas e colunas que não existem. Para o layout existem o Flexbox e o Grid (Capítulo 5).

2.10 HTML semântico

Um HTML é **semântico** quando cada elemento foi escolhido pelo que o conteúdo **é**, e não pela aparência. Para agrupar conteúdo sem nenhum significado especial, existem o `<div>`, para blocos, e o `<span>`, para trechos dentro de uma linha, quase sempre com uma `class`, que o CSS vai usar:

```
<div class="cartao">
  <p>Promoção: <span class="preco">R$ 15</span> a fatia.</p>
</div>
```

Eles são úteis, mas, para as grandes regiões da página, o HTML tem elementos próprios, que dizem o papel de cada uma:

Elemento	O que representa
`<header>`	o cabeçalho: o nome do site, o logotipo, a busca
`<nav>`	um bloco de links de navegação
`<main>`	o conteúdo principal da página; só um por página
`<article>`	um conteúdo completo em si, que faria sentido sozinho: uma receita, uma notícia, um comentário
`<section>`	uma seção temática, quase sempre com um título
`<aside>`	um conteúdo relacionado, mas à parte: dicas, anúncios, links relacionados
`<footer>`	o rodapé: direitos, contatos, links institucionais

O começo e o fim da página de receita usam esses elementos:

```
<header>
  <p><a href="index.html">Receitas da Casa</a></p>
  <nav aria-label="Principal">
    <ul>
      <li><a href="bolos.html">Bolos</a></li>
```

```
      <li><a href="salgados.html">Salgados</a></li>
      <li><a href="contato.html">Contato</a></li>
    </ul>
  </nav>
</header>
```

O `aria-label` dá um nome ao menu: se a página tiver mais de um `<nav>`, o leitor de tela consegue distinguir o "Principal" de outros. O conteúdo vem em seguida, dentro do `<main>`:

```
<main>
  <article>
    <h1>Bolo de cenoura com cobertura de chocolate</h1>
    <p>Publicada em <time datetime="2026-10-12">12 de outubro
      de 2026</time>. Rende <strong>12 fatias</strong>.</p>
```

E, depois do artigo, a dica e o rodapé:

```
    </article>

    <aside>
      <h2>Dica</h2>
      <p>Para a cobertura, veja a receita de
        <a href="calda.html">calda de chocolate</a>.</p>
    </aside>
  </main>

  <footer>
    <p><small>&copy; 2026 Receitas da Casa.</small></p>
  </footer>
</body>
</html>
```

Para ver a diferença que isso faz, a mesma página foi escrita do jeito errado, só com `<div>`, como ainda se encontra em sites antigos. Este é o conteúdo dela:

```
  <div class="topo">
    <div class="logo">Receitas da Casa</div>
    <div class="menu">
      <div onclick="location.href='bolos.html'">Bolos</div>
      <div onclick="location.href='salgados.html'">Salgados</div>
    </div>
  </div>
  <div class="conteudo">
    <div class="titulo">Bolo de cenoura</div>
    <img src="img/bolo.svg" width="320" height="200">
    <div class="subtitulo">Ingredientes</div>
    <div>3 cenouras médias</div>
    <div>3 ovos</div>
  </div>
```

Na tela, com algum CSS, as duas versões poderiam ficar idênticas. Por baixo, não. As duas foram conferidas da mesma forma:

Conferência	Página semântica	Só com `<div>`
validador do W3C	sem erros	erro: `<img>` sem `alt`
verificador de acessibilidade (axe)	nenhuma violação	4 violações
navegar com a tecla `Tab`	passa pelos 5 links	não chega a nenhum item do menu

O último resultado é o mais grave. Quem não usa mouse navega com a tecla `Tab`, que salta de um elemento interativo para o próximo: links, botões, campos. O `<a>` é interativo por natureza; um `<div>` com `onclick` não é, e o menu simplesmente não existe para essas pessoas. As quatro violações do axe foram a imagem sem texto alternativo, a falta de um `<main>`, a falta de um `<h1>` e conteúdo fora de qualquer região da página.

2.11 Acessibilidade

Acessibilidade é a possibilidade de qualquer pessoa usar o site, inclusive pessoas cegas, que ouvem a página por um **leitor de tela**, como o NVDA, gratuito, ou o leitor que vem no celular; pessoas com baixa visão, que ampliam a tela; pessoas com deficiência motora, que usam só o teclado ou comandos de voz; pessoas surdas, que dependem de legendas; e pessoas com deficiência cognitiva, que precisam de um conteúdo claro. Segundo o Censo de 2022 do IBGE, são cerca de 14 milhões de brasileiros com algum tipo de deficiência, sem contar quem tem uma limitação temporária, como um braço quebrado, ou circunstancial, como o sol forte na tela do celular.

No Brasil, a acessibilidade dos sites é também obrigação legal: a Lei Brasileira de Inclusão (Lei n.º 13.146/2015), no artigo 63, exige sites acessíveis de empresas com sede ou representação no país e de órgãos de governo. A referência técnica internacional são as **WCAG** (*Web Content Accessibility Guidelines*) do W3C, hoje na versão 2.2, e o nível que costuma ser exigido é o AA.

A boa notícia é que a maior parte da acessibilidade vem de graça com o HTML semântico. O leitor de tela não "vê" a página: ele percorre uma **árvore de acessibilidade**, que o navegador monta a partir do HTML, com o papel e o nome de cada elemento. Este é o começo dessa árvore para a página de receita, obtido do navegador com o Playwright, uma ferramenta que controla navegadores por código:

```
- banner:
  - paragraph:
    - link "Receitas da Casa"
  - navigation "Principal":
    - list:
      - listitem:
        - link "Bolos"
      ...
- main:
  - article:
    - heading "Bolo de cenoura com cobertura de chocolate"
      [level=1]
```

```
    - figure "O bolo pronto, ainda na forma.":
      - img "Bolo retangular com cobertura de chocolate"
    - heading "Ingredientes" [level=2]
    - list:
      - listitem: 3 cenouras médias
      ...
```

Cada elemento semântico virou um ponto de referência com nome: `banner` é o `<header>`, `navigation` é o `<nav>`, `main`, os títulos com o seu nível, a imagem com o seu `alt`. Com isso, a pessoa pode pular direto para o conteúdo principal, ouvir a lista de títulos ou a lista de links. Para a versão só com `<div>`, a árvore inteira foi esta:

```
- text: Receitas da Casa Bolos Salgados Bolo de cenoura
- img
- text: Ingredientes 3 cenouras médias 3 ovos
```

Um texto corrido, sem títulos, sem menu e com uma imagem sem nome. Uma lista do que conferir em toda página:

- o idioma no `<html lang>`;
- um `<h1>` e títulos em ordem, sem pular níveis;
- as regiões marcadas: `<header>`, `<nav>`, `<main>`, `<footer>`;
- texto alternativo em todas as imagens, vazio nas decorativas;
- links com texto que diz o destino;
- tudo o que é clicável feito com `<a>` ou `<button>`, e alcançável pelo teclado;
- legendas nos vídeos;
- contraste suficiente entre texto e fundo (Capítulo 4).

Existem também os atributos **ARIA**, como o `aria-label` do menu, que acrescentam informações à árvore de acessibilidade. A primeira regra para usá-los, escrita pelo próprio W3C, é: se existe um elemento HTML que faz o que você precisa, use o elemento, e não o ARIA. Um `<button>` já é um botão para todos; um `<div role="button">` precisa de muito trabalho extra para chegar perto disso. Para conferir uma página, a extensão **axe DevTools**, gratuita, e a aba **Lighthouse** das ferramentas de desenvolvedor do Chrome apontam os problemas mais comuns. Elas encontram parte dos problemas, mas não todos: vale também navegar pela página só com o teclado.

2.12 Caracteres especiais

Alguns caracteres têm significado no HTML, como o `<`, que abre uma tag. Para mostrá-los no texto, usam-se as **entidades**, que começam com `&` e terminam com `;`:

```
<p>Use &lt;p&gt; para parágrafos.</p>
<p>Arroz &amp; feijão custam R$&amp;nbsp;20.</p>
<p>&amp;copy; 2026 &amp;mdash; todos os direitos reservados.</p>
```

Entidade	Mostra	Uso
`<` e `>`	< e >	mostrar código HTML no texto
`&`	&	o “e comercial”
` `	(espaço)	um espaço que não quebra a linha, como entre “R$” e o valor
`©`	©	o símbolo de direitos autorais
`—`	—	o travessão

Com o UTF-8, letras acentuadas e símbolos como © podem ser digitados diretamente; as entidades são necessárias mesmo só para o <, o >, o & e os caracteres invisíveis, como o espaço que não quebra. Também existem entidades numéricas, pelo código do caractere: o `‑` é um hífen que não quebra a linha. O html-validate pediu os dois no link de telefone da Seção 2.5, `(21) 99999‑0000`, para que o número nunca seja partido entre duas linhas.

2.13 A página completa

Juntando os blocos das seções anteriores, a página de receita fica assim no navegador, ainda sem nenhum CSS, só com o estilo padrão:



Receitas da Casa

- Bolos
- Salgados
- Contato

Bolo de cenoura com cobertura de chocolate

Publicada em 12 de outubro de 2026. Rende **12 fatias**.

O bolo pronto, ainda na forma.

Ingredientes

- 3 cenouras médias
- 3 ovos
- 1 xícara de óleo
- 2 xícaras de açúcar
- 2 xícaras de farinha de trigo
- 1 colher (sopa) de fermento em pó

Modo de preparo

1. Bata no liquidificador as cenouras, os ovos e o óleo.
2. Misture o açúcar e a farinha, e por último o fermento.
3. Asse em forno médio, a 180 °C, por cerca de 40 minutos.

Não abra o forno nos primeiros 25 minutos.

Informação nutricional

Valores aproximados por fatia

Item	Quantidade
Calorias	310 kcal
Carboidratos	45 g

Dica

Para a cobertura, veja a receita de calda de chocolate.

© 2026 Receitas da Casa.

O navegador já dá alguma forma à página: títulos maiores, marcadores nas listas, números no modo de preparo, links sublinhados. Isso é o estilo padrão, que cada navegador aplica, e que o CSS vai substituir a partir do Capítulo 4. A página foi conferida pelos dois validadores, o do W3C e o html-validate, sem nenhum erro; pelo axe, sem nenhuma violação; e pela navegação com o teclado. O navegador baixou o HTML, a imagem do bolo e o *favicon*, todos com o status `200`.

Erros Comuns

- **Esquecer o `alt` das imagens.** A imagem some para quem usa leitor de tela. Descreva o que ela mostra, ou use `alt=""` se for decorativa.
- **Escolher o título pelo tamanho da letra.** Os níveis de título formam o sumário da página. Escolha pela hierarquia e ajuste o tamanho com o CSS.
- **Fazer botões e menus com `<div>`.** Eles não recebem o foco do teclado nem são anunciados como clicáveis. Use `<a>` e `<button>`.
- **Fechar os elementos fora de ordem.** `<p><strong>...</p></strong>` confunde o navegador. Feche primeiro o que abriu por último.
- **Usar `<br>` para dar espaço.** O espaçamento é trabalho do CSS.
- **Usar tabelas para posicionar a página.** Tabelas são para dados; o layout é feito com Flexbox e Grid.
- **Links com "clique aqui".** Fora de contexto, o texto não diz nada. Escreva o destino.
- **Esquecer o `lang` e o `charset`.** O leitor de tela lê com o sotaque errado, e os acentos podem virar símbolos estranhos.
- **Confiar só no que aparece na tela.** Uma página pode parecer certa e ter erros. Passe pelo validador e pelo verificador de acessibilidade.